Guia do Simulador RuFaS

Oct 28, 2026 · @Someone

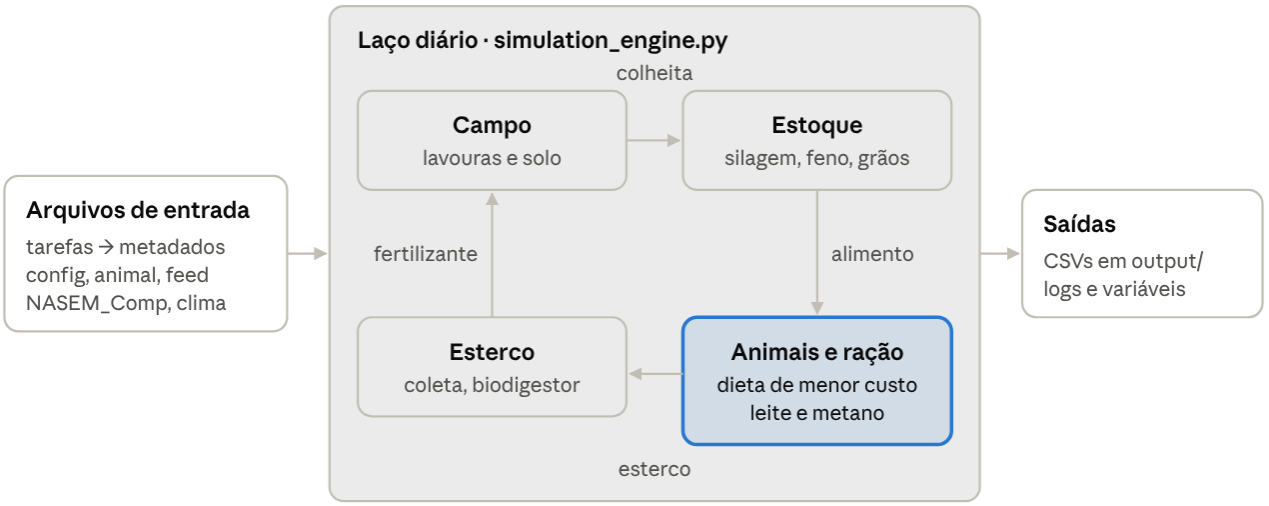
Visão geral

O RuFaS (Ruminant Farm Systems) simula, dia a dia, uma fazenda leiteira inteira: rebanho, ração, lavouras, solo, esterco e emissões. Cada execução lê um conjunto de arquivos JSON/CSV de entrada e grava os resultados em `output/`.

O simulador é dividido em módulos biofísicos, em `RUFAS/biophysical/`:

Módulo	Pasta	O que simula
Animal	`biophysical/animal/`	Rebanho, reprodução, crescimento, leite, exigências nutricionais (NASEM ou NRC), formulação de ração, metano entérico
Armazenamento de alimentos	`biophysical/feed_storage/`	Silagem, feno, grãos, pré-secado e compras de alimento
Campo	`biophysical/field/`	Lavouras, solo, fertilização, preparo do solo, colheita
Esterco	`biophysical/manure/`	Coleta, separação, biodigestor e armazenamento de esterco
EEE	`RUFAS/EEE/`	Economia, energia e emissões (custos, fatores de emissão)

Os módulos conversam entre si: a colheita abastece o estoque de alimentos, o estoque alimenta a ração, os animais produzem leite e esterco, e o esterco volta ao campo como fertilizante.



fluxo diário entre os módulos

A ração é reformulada a cada `formulation_interval` dias; os demais passos rodam todo dia, na ordem campo, estoque, ração, animais e esterco.

A orquestração fica em três arquivos: `RUFAS/task_manager.py` (lê a lista de tarefas e as executa, inclusive em paralelo), `RUFAS/input_manager.py` (carrega e valida as entradas) e `RUFAS/simulation_engine.py` (laço diário da simulação).

Instalação e execução

Com Python 3.12 ou 3.13, quatro comandos rodam o exemplo padrão (fazenda freestall no Wisconsin, 2013–2019):

```
python -m venv venv
source venv/bin/activate
pip install .
python main.py
```

Antes da primeira execução, renomeie `output/output_filters/_csv_all_variables.txt` para `csv_all_variables.txt` (sem o sublinhado). Sem isso, nenhum CSV de resultados é gravado.

Como uma execução é encadeada

1. `main.py` lê o arquivo de tarefas, por padrão `input/task_manager_metadata.json`.
2. Esse arquivo aponta para uma lista de tarefas, como `input/data/tasks/example_freestall_task.json`.

3. Cada tarefa aponta para um arquivo de metadados (`input/metadata/*.json`), que lista todos os arquivos de dados da fazenda.
4. O simulador roda e grava os resultados em `output/` .

Argumentos de linha de comando

Opção	Padrão	Efeito
`-p` , `--path-to-metadata`	`input/task_manager_metadata.json`	Arquivo de tarefas a executar
`-o` , `--output-dir`	`output/`	Pasta de resultados
`-l` , `--logs-dir`	`output/logs`	Pasta de logs
`-v` , `--verbosity`	definido na tarefa	`errors` , `warnings` , `logs` , `credits` ou `none`
`-c` , `--clear-output`	desligado	Apaga a pasta de saída antes de rodar
`-g` , `--no-graphics`	desligado	Não gera gráficos
`-s` , `--suppress-log-files`	desligado	Não grava arquivos de log
`-i` , `--exclude_info_maps`	desligado	Remove os info_maps (metadados de origem) da saída
`-m` , `--metadata-depth-limit`	—	Muda o limite de profundidade dos metadados

Tipos de tarefa

O campo `task_type` de cada tarefa escolhe o que é executado. Os mais úteis:

- `SIMULATION_SINGLE_RUN` : uma simulação.
- `SIMULATION_MULTI_RUN` : várias simulações trocando a semente aleatória.
- `SENSITIVITY_ANALYSIS` : análise de sensibilidade sobre parâmetros de entrada.
- `HERD_INITIALIZATION` : gera um rebanho estável para iniciar simulações.
- `INPUT_DATA_AUDIT` : só valida as entradas e exporta as propriedades em CSV.
- `POST_PROCESSING` : reprocessa saídas sem simular de novo.

Os demais (`END_TO_END_TESTING` , `UPDATE_E2E_TEST_RESULTS` , `COMPARE_METADATA_PROPERTIES` , `DATA_COLLECTION_APP_UPDATE`) servem ao desenvolvimento do próprio RuFaS.

Estrutura do repositório

O código está em `RUFAS/` ; os dados de entrada estão em `input/` ; os resultados vão para `output/` .

Caminho	Conteúdo
`main.py`	Ponto de entrada; chama `RUFAS/main.py`
`RUFAS/biophysical/`	Modelos biofísicos: `animal/` , `feed_storage/` , `field/` , `manure/`
`RUFAS/biophysical/animal/nutrients/`	Exigências (NASEM e NRC) e oferta de nutrientes da ração
`RUFAS/biophysical/animal/ration/`	Formulação e otimização de ração (`ration_optimizer.py`)
`RUFAS/EEE/`	Economia, energia e emissões
`RUFAS/data_structures/`	Objetos trocados entre módulos (ex.: `feed_storage_to_animal_connection.py` , que define `Feed` e `NutrientStandard`)
`RUFAS/input/metadata/properties/`	Esquemas que validam cada arquivo de entrada (tipos, limites, valores padrão)
`RUFAS/post_processing/` e `RUFAS/output_manager.py`	Coleta de variáveis, filtros, relatórios e gráficos
`input/task_manager_metadata.json`	Arquivo de tarefas padrão
`input/metadata/`	Metadados de cada cenário de fazenda
`input/data/`	Dados: animal, feed, crop, soil, weather, manure, EEE etc.
`output/output_filters/`	Filtros que definem quais variáveis são salvas
`docs/scientific/`	PDFs com a documentação científica de cada módulo (`animalmoddoc.pdf` , `cropandsoilmoddoc.pdf` , `feedstoragemoddoc.pdf` , `manuremoddoc.pdf` , `main.pdf`)

Caminho	Conteúdo
`docs/_src/_wiki/`	Wiki técnica (Task Manager, Input Manager, Output Manager etc.), também publicada no GitHub Pages
`DataCollectionApp/`	Interface web para montar arquivos de entrada
`tests/`	Testes unitários e ponta a ponta

Para entender a ciência por trás de cada cálculo, comece por `docs/scientific/animalmoddoc.pdf`; ele descreve as equações de exigência e a formulação de ração.

Entradas e a biblioteca de alimentos

Não existe um arquivo chamado exatamente `NASEM_Comp.csv`. `NASEM_Comp` é a **chave** usada nos metadados; o arquivo que ela aponta é `input/data/feed/NASEM_Comp_with_TDN_urea.csv`.

O arquivo de metadados é o mapa de tudo

Cada cenário de fazenda tem um arquivo em `input/metadata/` (o padrão é `example_freestall_dairy_metadata.json`). Ele lista, sob `files`, cada entrada com uma chave, um `path` e um esquema de validação (`properties`). Para trocar um dado, aponte a chave para outro arquivo; o código acessa os dados pela chave, por exemplo `config.nutrient_standard`.

Chave	Arquivo no exemplo freestall	Conteúdo
`config`	`input/data/config/example_freestall_config.json`	Datas de início e fim, padrão nutricional (`NASEM` ou `NRC`), condado (FIPS), semente aleatória

Chave	Arquivo no exemplo freestall	Conteúdo
`animal`	`input/data/animal/example_freestall_animal.json`	Tamanho do rebanho, manejo reprodutivo, baias, parâmetros de ração
`feed`	`input/data/feed/example_Midwest_feed.json`	Alimentos disponíveis, preços e rações por grupo animal
`NASEM_Comp`	`input/data/feed/NASEM_Comp_with_TDN_urea.csv`	Composição nutricional dos alimentos, padrão NASEM (2021)
`NRC_Comp`	`input/data/feed/NRC_comp.csv`	Composição nutricional, padrão NRC (2001)
`user_feeds`	`input/data/feed/user_feeds.csv`	Índice dos alimentos: ID, nome e em qual biblioteca cada um existe
`weather`	`input/data/weather/example_temperate_weather.csv`	Clima diário
`field`, `soil`, `crop`, schedules	`input/data/field/`, `soil/`, `crop/`, `*_schedule/`	Talhões, solo, rotação de culturas, fertilização, esterco e preparo

Chave	Arquivo no exemplo freestall	Conteúdo
`manure_management`	`input/data/manure/example_freestall_processor_configs.json`	Equipamentos de manejo de esterco e suas conexões
feed storage	`input/data/feed_management/`	Silos e armazéns que recebem a colheita
EEE	`input/data/EEE/`	Custos, fatores de emissão, tratores

Há cenários prontos em `input/metadata/`: só animais, só alimentação, só campo, só esterco, campo + alimentação, freestall completo e open lot.

A biblioteca NASEM

`NASEM_Comp_with_TDN_urea.csv` tem 176 alimentos e uma linha por alimento. A coluna `rufas_id` é o identificador usado em todos os outros arquivos.

Grupo de colunas	Exemplos	Unidade
Identificação	`rufas_id`, `Name`, `Fd_Category`, `feed_type` (Forage, Conc, Mineral)	—
Matéria seca e fibra	`DM`, `NDF`, `ADF`, `NDF48`, `lignin`, `ash`	% (fibras em % da MS)
Proteína	`CP`, `RUP`, `dRUP`, `N_A`, `N_B`, `N_C`, `Kd`, `ADICP`, `NDICP`	% da MS ou % da PB
Carboidratos e gordura	`starch`, `WSC`, `FA`, `EE`, ácidos graxos `C120_FA` a `C183_FA`	% da MS
Energia	`DE_Base`, `DE` (Mcal/kg), `TDN` (% da MS)	—
Minerais	`calcium`, `phosphorus`, `magnesium`, `potassium`, `sodium`, `copper`, `zinc` etc.	% ou mg/kg da MS
Aminoácidos	`lysine`, `methionine`, `histidine` etc.	% da PB

Grupo de colunas	Exemplos	Unidade
Vitaminas e absorção	`Vit_A`, `Vit_D`, `Vit_E`, `Abs_calcium`, `Abs_phosphorus` etc.	—
Limites de inclusão	`limit`, `lower_limit`	kg/animal/dia

As descrições, unidades e faixas válidas de cada coluna estão em `RUFAS/input/metadata/properties/default.json`, na seção `NASEM_Comp_properties`. A versão `NASEM_Comp_with_TDN.csv` é igual, sem a ureia (ID 307).

Como um alimento entra na simulação

1. Escolha o `rufas_id` em `user_feeds.csv` (a coluna `Comp_location` diz se ele existe em NASEM, NRC ou ambos).
2. Declare-o em `feeds` no arquivo `feed` (ex.: `example_Midwest_feed.json`), com `feed_type` = `rufas_id` e `purchased_feed_cost` em US$/kg de MS.
3. Em `rations`, defina a participação de cada alimento (`ration_percentage`) por grupo: `calf`, `growing`, `close_up`, `lac_cow`.
4. O simulador busca a composição na biblioteca do padrão escolhido em `config.nutrient_standard` (código: `AvailableFeedsBuilder` em `RUFAS/data_structures/feed_storage_to_animal_connection.py`).

No exemplo Midwest, a vaca em lactação recebe 37,45% de silagem de milho imatura (ID 50), 17,65% de silagem de leguminosa (ID 110), 15,31% de milho grão (ID 44), 25,21% de uma mistura de subprodutos (ID 302), 3,27% de mistura mineral (ID 301) e pequenas porções dos IDs 23 e 104.

Saídas: onde estão os dados de simulação

Os resultados ficam num único CSV largo em `output/CSVs/`, uma coluna por variável e uma linha por dia. Isso foi verificado rodando o exemplo freestall por 1 ano (2013): a execução levou cerca de 50 s e gerou 2.482 colunas e 97 MB.

Arquivos gerados

Arquivo	Pasta	Para que serve
`<prefixo>_saved_variables_<filtro>_<data>.csv`	`output/CSVs/`	Os dados da simulação, um arquivo por filtro `csv_*`

Arquivo	Pasta	Para que serve
`<prefixo>_variable_names_<data>.txt`	`output/logs/`	Lista de todas as variáveis disponíveis; use para montar filtros
`<prefixo>_variables_reported_daily_<data>.csv`	`output/logs/`	Quais variáveis têm valor diário
`<prefixo>_errors_`, `_warnings_`, `_logs_<data>.json`	`output/logs/`	Erros, avisos e mensagens da execução

O `<prefixo>` vem de `output_prefix` na tarefa (no exemplo, `freestall`). As pastas podem ser trocadas na tarefa com `csv_output_directory`, `json_output_directory`, `report_directory` e `filters_directory`.

Como ler as colunas

Cada coluna se chama `Classe.função.variável (unidade)`. Variáveis diárias têm 365 valores por ano; parâmetros fixos têm 1 ou 2 valores e o resto da coluna vazio. Exemplos úteis para um otimizador:

Coluna	O que mede
`AnimalModuleReporter.report_herd_statistics_data.daily_milk_production (kg/day)`	Leite do rebanho por dia (média de 3.663 kg/dia no teste de 1 ano)
`AnimalModuleReporter.report_ration_per_animal.ration_per_animal_for_LAC_COW_PEN_3.dry_matter_intake_total (kg)`	Consumo de matéria seca por vaca em lactação
`AnimalModuleReporter.report_enteric_methane_emission.enteric_methane_emission_for_LAC_COW_PEN_3 (g)`	Metano entérico da baia de lactação
`FeedManager.purchase_feed.ration_interval_<id>_cost ($)`	Custo de compra do alimento `<id>`
`RationManager.set_user_defined_rations.user_defined_ration.<id> (percent)`	Participação do alimento `<id>` na ração definida

Leitura em Python:

```
import pandas as pd

df =
pd.read_csv("output/CSVs/freestall_saved_variables_csv_all_variables.txt_<dat
a>.csv", low_memory=False)
leite =
df["AnimalModuleReporter.report_herd_statistics_data.daily_milk_production
(kg/day)"].dropna()
```

Filtros: salvar só o que interessa

O que é salvo depende dos arquivos em `output/output_filters/`. O prefixo do nome define o destino:

- `csv_*.txt` ou `json_*.txt`: uma expressão regular por linha; as variáveis cujo nome casar são salvas. O arquivo incluído, `_csv_all_variables.txt`, contém só `.*` (tudo).
- `report_*.json`: relatórios agregados (Report Generator).
- `graph_*.json`: gráficos (Graph Generator).

Arquivos que começam com `_` são ignorados. Para um otimizador, crie um filtro pequeno, como `csv_otimizador.txt` com as linhas `.*daily_milk_production.*` e `.*enteric_methane.*`; isso reduz o tempo de gravação e o tamanho do arquivo. Detalhes em `docs/_src/_wiki/Output-Manager.rst` e `Report-Generator.rst`.

Formulação de ração

O RuFaS já tem um otimizador: a cada `formulation_interval` dias (30 no exemplo), cada baia resolve um problema não linear que **minimiza o custo da ração** sujeito às exigências nutricionais da baia. O código está em `RUFAS/biophysical/animal/ration/ration_optimizer.py`.

O problema de otimização

- **Variável de decisão:** kg de matéria seca de cada alimento disponível, por animal por dia.
- **Objetivo:** `RationOptimizer.objective` = soma de (kg de cada alimento × `purchased_feed_cost`).
- **Limites:** `lower_limit` e `limit` de cada alimento na biblioteca NASEM/NRC (kg/animal/dia). Com ração definida pelo usuário, os limites viram a participação informada ± `user_defined_ration_tolerance`.

- **Restrições NASEM:** energia líquida, cálcio, fósforo, proteína mínima e máxima, FDN mínimo e máximo, FDN de forragem, gordura, consumo de MS mínimo e máximo. O NRC troca a restrição única de energia por quatro (total, manutenção, lactação, crescimento).
- **Solver:** `scipy.optimize.minimize` com `method="SLSQP"`. O ponto inicial é a ração anterior ou, na primeira vez, valores aleatórios.

De onde vêm as exigências e a oferta

Arquivo em `RUFAS/biophysical/animal/nutrients/`	Papel
`nasem_requirements_calculator.py`	Exigências pelas equações NASEM (2021)
`nrc_requirements_calculator.py`	Exigências pelas equações NRC (2001)
`nutrition_supply_calculator.py`	Nutrientes fornecidos por uma combinação de alimentos
`nutrition_evaluator.py`	Compara oferta e exigência

Quando a otimização falha

A lógica de novas tentativas está em `Pen._run_formulation_attempts` (`RUFAS/biophysical/animal/pen.py`). Só a baia de vacas em lactação tenta de novo; as outras ficam com o resultado da primeira tentativa.

1. Se a ração é do usuário e a falha é de uma restrição que mais consumo resolveria, o consumo de MS-alvo sobe e o solver roda de novo.
2. Senão, a exigência de produção de leite é reduzida, até `milk_reduction_maximum` (0,5 kg no exemplo).
3. O laço para após `maximum_ration_reformulation_attempts` (250 no exemplo).
4. Se ainda falhar com ração do usuário, a ração informada é usada como está. O resumo das restrições violadas sai na variável `failed_constraint_summary_for_pen_<id>`.

Ração do usuário ou ração otimizada

O campo `ration.user_input` no arquivo `animal` decide o modo. Com `true` (o padrão do exemplo), o solver só ajusta as participações dentro da tolerância em torno de `rations`. Com `false`, ele escolhe livremente entre os alimentos disponíveis para cada grupo.

Como plugar um otimizador

Há dois caminhos: tratar o RuFaS como caixa-preta (mudar entradas, rodar, ler o CSV) ou alterar o otimizador de ração interno. O primeiro não exige mexer no código e serve para qualquer decisão da fazenda; o segundo atua só na dieta, mas dentro de cada formulação.

Caminho 1: otimizador externo (caixa-preta)

Cada avaliação da função objetivo é uma simulação completa: cerca de 50 s por ano simulado no exemplo freestall, numa máquina comum. Prefira métodos econômicos em avaliações (otimização bayesiana, superfície de resposta, algoritmos evolutivos com população pequena) e simulações curtas durante a busca.

1. Copie o cenário (tarefa, metadados e os JSON que vão variar) para uma pasta de trabalho.
2. Escreva os valores candidatos nos JSON copiados, por exemplo `ration_percentage` e `purchased_feed_cost` no arquivo `feed`, ou `herd_information.cow_num` no arquivo `animal`.
3. Aponte os metadados copiados para esses arquivos e rode `python main.py -p <tarefas.json> -o <pasta> -g`.
4. Leia o CSV de saída e calcule o objetivo (leite, custo de alimento, metano, N no esterco etc.).

Esboço em Python:

```
import json, subprocess, glob
import pandas as pd

def avaliar(ration_lac_cow: dict[int, float], pasta: str) -> float:
    feed = json.load(open("input/data/feed/example_Midwest_feed.json"))
    for r in feed["rations"]:
        if r["animal_combination"] == "lac_cow":
            r["feeds"] = [{"feed_type": i, "ration_percentage": p} for i, p
in ration_lac_cow.items()]
    json.dump(feed, open(f"{pasta}/feed.json", "w"))
    # metadados e tarefa copiados apontam para {pasta}/feed.json e gravam em
{pasta}/out
    subprocess.run(["python", "main.py", "-p", f"{pasta}/tm.json", "-o", f"
{pasta}/out", "-g", "-c"], check=True)
    df = pd.read_csv(glob.glob(f"{pasta}/out/CSVs/*.csv")[0],
low_memory=False)
    leite =
df["AnimalModuleReporter.report_herd_statistics_data.daily_milk_production
(kg/day)"].sum()
    custo =
df.filter(like="FeedManager.purchase_feed").filter(like="_cost").sum().sum()
    return custo / leite  # US$ de alimento por kg de leite
```

Na tarefa, `csv_output_directory`, `json_output_directory` e `filters_directory` definem para onde cada execução grava; use uma pasta por candidato para rodar em paralelo.

Amostragem embutida: SENSITIVITY_ANALYSIS

Para explorar o espaço antes de otimizar, a tarefa `SENSITIVITY_ANALYSIS` já amostra entradas e roda as simulações. Cada variável é nomeada pelo caminho no pool de entrada, como em `input/data/tasks/sample_tasks.json`:

```
{
  "task_type": "SENSITIVITY_ANALYSIS",
  "metadata_file_path":
"input/metadata/example_freestall_dairy_metadata.json",
  "sampler": "sobol",
  "sampler_n": 64,
  "SA_input_variables": [
    {"variable_name": "animal.herd_information.cow_num", "lower_bound": 90,
"upper_bound": 110, "data_type": "int"}
  ]
}
```

Os amostradores vêm da biblioteca SALib (`sobol` , `fractional_factorial` e `morris`). Junto com `parallel_workers` , isso gera rapidamente um conjunto de treino para um modelo substituto (surrogate).

Caminho 2: mudar o otimizador de ração interno

Para otimizar a dieta com outro objetivo, os pontos de entrada são:

- `RationOptimizer.objective` : hoje é só custo. Dá para somar termos como metano ou N urinário; a `RationConfig` já carrega `pen_average_enteric_methane` e `pen_average_urine_nitrogen` .
- `RationOptimizer.set_constraints` : acrescente ou remova restrições (cada uma é uma função que retorna ≥ 0 quando satisfeita).
- `RationOptimizer.attempt_optimization` : chamada de `scipy.optimize.minimize` ; troque aqui o solver.
- `Pen._run_formulation_attempts` (`pen.py`): o que acontece quando o solver falha.

Os testes desse módulo ficam em `tests/test_biophysical/test_animal/` ; rode `pytest tests/test_biophysical/test_animal` após mudar algo.

Referência rápida

Preciso de…	Onde
Rodar o exemplo padrão	`python main.py`
Rodar outra lista de tarefas	`python main.py -p <task_manager_metadata.json> -o <saída> -g`
Mudar datas, padrão nutricional ou condado	`input/data/config/*.json`
Composição nutricional NASEM	`input/data/feed/NASEM_Comp_with_TDN_urea.csv` (chave `NASEM_Comp`)
Composição nutricional NRC	`input/data/feed/NRC_comp.csv` (chave `NRC_Comp`)
Nome de um alimento pelo ID	`input/data/feed/user_feeds.csv`
Alimentos, preços e rações	`input/data/feed/example_*_feed.json`
Rebanho e parâmetros de ração	`input/data/animal/example_*_animal.json`
Significado e unidade de qualquer campo de entrada	`RUFAS/input/metadata/properties/default.json`

Preciso de…	Onde
Campos de uma tarefa	`RUFAS/input/metadata/properties/tasks_properties.json`
Resultados	`output/CSVs/<prefixo>_saved_variables_*.csv`
Lista de variáveis de saída	`output/logs/<prefixo>_variable_names_*.txt`
Escolher o que salvar	`output/output_filters/csv_*.txt`
Otimizador de ração	`RUFAS/biophysical/animal/ration/ration_optimizer.py`
Equações científicas	`docs/scientific/animalmoddoc.pdf` e demais PDFs
Wiki técnica	`docs/_src/_wiki/` ou GitHub Pages